

Java Collections Framework

Juan Ignacio Raggio

May 3, 2025

Contents

1	Coleccion	1
2	Interfaz Collection	2
2.0.1	Importante:	3
2.0.2	Iteradores de colecciones	4
2.1	Set:	4
2.2	SequencedCollection	4
2.2.1	reversed()	5
3	Interfaz Set	5
3.1	HashSet: Hash table	5
4	Interfaz SortedSet	6
4.1	TreeSet: Arbol	6
4.2	EnumSet:	6
4.3	Queue	6
5	Siempre retornar el valor mas generico posible	6

1 Coleccion

- Es un objeto que agrupa a un conjunto de `elementos` compatibles
- Las colecciones son iterables por lo que todas extienden indirectamente a `Iterable`
-

2 Interfaz Collection

- Es el minimo denominador comun entre todas las colecciones implementadas - Extiende iterable

```
public interface Collection<E> extends Iterable<E> {
    int size();

    boolean isEmpty();

    boolean contains(Object o);

    Iterator<E> iterator();

    Object[] toArray();

    <T> T[] toArray(T[] a);

    default <T> T[] toArray(IntFunction<T[]> generator) {
        return toArray(generator.apply(0));
    }

    boolean add(E e);

    boolean remove(Object o);

    boolean containsAll(Collection<?> c);

    boolean addAll(Collection<? extends E> c); // es una union de colecciones

    boolean removeAll(Collection<?> c); // Resta de colecciones

    default boolean removeAll(Predicate<? super E> filter) {
        Objects.requireNonNull(filter);
        boolean removed = false;
        final Iterator<E> each = iterator();
        while (each.hasNext()) {
            if (filter.test(each.next())) {
                each.remove();
                removed = true;
            }
        }
    }
}
```

```

        }
    }
    return removed;
}

boolean retainAll(Collection<?> c); // Interseccion de colecciones

void clear();

boolean equals(Object o);

int hashCode();

@Override
default Spliterator<E> spliterator() {
    return Spliterators.spliterator(this, 0);
}

default Stream<E> stream() {
    return StreamSupport.stream(spliterator(), false);
}

default Stream<E> parallelStream() {
    return StreamSupport.stream(spliterator(), true);
}
}

```

- Cada coleccion va a implementar add, addAll, etc, segun lo que se espera de la misma
- Show all - Muestra todos los elementos separados por una coma

2.0.1 Importante:

- Si quiero usar **contains**, **sobrecribir equals()** en las clases que quiero usar como el “generico” de la coleccion
- Tengo que saber que metodos tengo que sobrecribir
- Una coleccion **NO se puede modificar durante una iteracion**

2.0.2 Iteradores de colecciones

- Tienen una complejidad mayor a los iteradores que implementamos nosotros
- Soportan multithreading -> Por eso intentar modificar una coleccion en un loop nos tira una `Exception ConcurrentModificationException`

2.1 Set:

-

2.2 SequencedCollection

- Familia de colecciones que permite ver las colecciones con un **orden secuencial**. Ejemplo:
 - Dame el elemento 1
 - Agrega un elemento al inicio
 - Agrega en la posicion i

```
public interface SequencedCollection<E> extends Collection<E> {

    SequencedCollection<E> reversed();

    default void addFirst(E e) {
        throw new UnsupportedOperationException();
    }

    default void addLast(E e) {
        throw new UnsupportedOperationException();
    }

    default E getFirst() {
        return this.iterator().next();
    }

    default E getLast() {
        return this.reversed().iterator().next();
    }
}
```

```

default E removeFirst() {
    var it = this.iterator();
    E e = it.next();
    it.remove();
    return e;
}

default E removeLast() {
    var it = this.reversed().iterator();
    E e = it.next();
    it.remove();
    return e;
}
}

```

2.2.1 reversed()

-

3 Interfaz Set

3.1 HashSet: Hash table

- Estrategia para almacenamiento y recuperacion de informacion asociada a una clave unica
- En el caso ideal todas las operaciones son de $O(1)$
- En el peor caso $O(n)$
- Elementos: Pair $\rightarrow$ (key, value)
- A cada clave se le aplica una funcion (hash) que genera un valor numerico. El indice del arreglo se calcula como :

$H(\text{key}) = \text{key.hash}$ Si hay colisiones se solucionan como **Separate Chaining** $\rightarrow$ Hacer una lista en

- La otra forma es aprovechando los espacios vacios que quedaron en la tabla

4 Interfaz SortedSet

- No se tienen que poder cambiar los elementos que definen el orden directamente
- Tengo que removerlos y luego agregarlos. Porque si no se cumple el orden puede que no se encuentre el elemento

4.1 TreeSet: Arbol

4.2 EnumSet:

4.3 Queue

5 Simpre retornar el valor mas generico posible

- Es una buena practica
- El dia de mañana si se desea cambiar una implementacion es sencillo
- Programar Contra interfaces

```
// Simpre poner la interfaz mas generica
public Iterable<SharedFile> getAllFiles() {
    // Para clonar un array list:
    return new ArrayList<>(files);
}
```